

CAA Lab Assignment 4

```
struct CPU {
    uint8_t R[4];
    uint8_t ACC;
    uint8_t PC;
    uint8_t Z, C;
};

struct IFEX {
    uint8_t IR;
    bool valid;
};

struct CacheLine {
    bool valid;
    uint8_t tag;
    uint8_t data;
};

CPU cpu;
IFEX ifex;
CacheLine cache[CACHE_LINES];
uint8_t IMEM[MEM_SIZE];
uint8_t DMEM[MEM_SIZE];

int cycles = 0;
int instructions = 0;
int stalls = 0;
int forwardings = 0;
int cache_hits = 0;
int cache_misses = 0;
bool jump_taken = false;

void reset_cpu() {
    memset(&cpu, 0, sizeof(cpu));
    memset(&ifex, 0, sizeof(ifex));
    memset(cache, 0, sizeof(cache));
}
```

```

void load_program(const string &fname) {
    ifstream fin(fname);
    if (!fin) {
        cout << "Cannot open program file\n";
        exit(1);
    }
    int addr = 0;
    unsigned int x;
    while (fin >> hex >> x)
        IMEM[addr++] = (uint8_t)x;
}

bool is_load(uint8_t ir) {
    return ((ir >> 4) & 0xF) == 0xD;
}

bool writes_register(uint8_t ir) {
    int op = (ir >> 4) & 0xF;
    return (op == 0x0 || op == 0x1 || op == 0x2 || op == 0x3 ||
            (op >= 0x4 && op <= 0x7));
}

uint8_t cache_read(uint8_t addr) {
    int index = addr % CACHE_LINES;
    uint8_t tag = addr / CACHE_LINES;

    if (cache[index].valid && cache[index].tag == tag) {
        cache_hits++;
        return cache[index].data;
    }

    cache_misses++;
    cycles += CACHE_MISS_PENALTY;

    cache[index] = {true, tag, DMEM[addr]};
    return DMEM[addr];
}

void cache_write(uint8_t addr, uint8_t val) {
    int index = addr % CACHE_LINES;
    uint8_t tag = addr / CACHE_LINES;

    DMEM[addr] = val;

```

```

    cache[index] = {true, tag, val};
}

uint8_t alu(uint8_t a, uint8_t b, int op) {
    int res = 0;
    if (op == 0x4) res = a + b;
    if (op == 0x5) res = a - b;
    if (op == 0x6) res = a * b;
    if (op == 0x7) res = (b == 0 ? 0 : a / b);

    cpu.Z = (res == 0);
    cpu.C = (res > 255);
    return res & 0xFF;
}

void execute_stage() {
    if (!ifex.valid) return;

    uint8_t ir = ifex.IR;
    int op = (ir >> 4) & 0xF;
    int operand = ir & 0xF;

    instructions++;

    switch (op) {
    case 0x1: cpu.ACC = cpu.R[operand]; break;
    case 0x2: cpu.R[operand] = cpu.ACC; break;
    case 0x3: cpu.ACC = operand; break;
    case 0x4: cpu.ACC = alu(cpu.ACC, cpu.R[operand], 0x4); break;
    case 0x5: cpu.ACC = alu(cpu.ACC, cpu.R[operand], 0x5); break;
    case 0x6: cpu.ACC = alu(cpu.ACC, cpu.R[operand], 0x6); break;
    case 0x7: cpu.ACC = alu(cpu.ACC, cpu.R[operand], 0x7); break;
    case 0x0: cpu.R[operand] = cache_read(operand); break;
    case 0xE: cache_write(operand, cpu.R[operand]); break;
    case 0xA:
        cpu.PC = operand;
        jump_taken = true;
        break;
    }
}

void fetch_stage() {
    ifex.IR = IMEM[cpu.PC++];
}

```

```
ifex.valid = true;  
}
```